Part 3: Security and The Approximate-GCD Problem

Speaker Notes & Mathematical Primer

Nirav Bhattad

March 27, 2026

§1. The Security Reduction

How to start speaking: "We have built a scheme that can homomorphically add and multiply, albeit with a noise bottleneck. But before we solve that bottleneck, we must ask: is this scheme actually secure?
In cryptography, we don't just guess that a scheme is secure; we prove that breaking it is equivalent to solving a known hard mathematical problem. For RSA, it's factoring. For Gentry's original FHE, it's finding short vectors in ideal lattices. For our integer scheme, the authors reduce the semantic security to a surprisingly simple-sounding problem: the Approximate Integer Greatest Common Divisor, or Approximate-GCD."

Recall our distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where the quotient q is chosen such that $x \approx 2^\gamma$, and the noise r is chosen from $(-2^\rho, 2^\rho)$.

Definition 1.1: The (ρ, η, γ)-Approximate-GCD Problem

Given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

The authors formally prove a search-to-decision reduction: if an attacker can predict the encrypted bit with a non-negligible advantage, they can be used as an oracle to extract the least-significant bit of quotients, which ultimately allows the recovery of the secret key p via a modified binary GCD algorithm. Therefore, semantic security holds as long as Approximate-GCD is hard.

§2. Why Standard Algorithms Fail

The Naive Attack: "Let's put on our attacker hats. We are given x_1 and x_2. If there were no noise $(r = 0)$, x_1 and x_2 would be exact multiples of p. We would simply run the 2,000-year-old Euclidean algorithm, compute $\gcd(x_1, x_2)$, and immediately recover p.
But the noise ruins everything. The Euclidean algorithm relies on strict divisibility. With even a tiny bit of noise, the remainders in the Euclidean algorithm explode rather than shrinking to zero.
What about Continued Fractions? They are great for finding rational approximations. Since $x_1/x_2 \approx q_1/q_2$, maybe q_1/q_2 appears in the continued fraction expansion of x_1/x_2? The math shows that the distance $|x_1/x_2 - q_1/q_2|$ is dominated by $(q_2 r_1 - q_1 r_2)/q_2^2$, which is not small enough to guarantee q_1/q_2

will be found. The noise completely obscures the target."

§3. Lattice Attacks and Parameter Justification

Because simple algebraic attacks fail, we must turn to the geometry of numbers, specifically lattice reduction algorithms like LLL. When an attacker is given many samples $x_0, x_1, \ldots, x_t$, the rational numbers $y_i = x_i/x_0$ form an instance of the Simultaneous Diophantine Approximation (SDA) problem.
We can attempt to use Lagarias' algorithm for SDA by applying lattice reduction to the $(t+1)$-dimensional lattice L spanned by the rows of the following matrix M:

$$M = \begin{pmatrix} 2^\rho & x_1 & x_2 & \ldots & x_t \\ 0 & -x_0 & 0 & \ldots & 0 \\ 0 & 0 & -x_0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & -x_0 \end{pmatrix} \tag{3.1}$$

The attacker wants to find the hidden quotients. Notice what happens if we multiply the matrix M by the vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$:

$$\vec{v} = \langle q_0, q_1, \ldots, q_t \rangle \cdot M \tag{3.2}$$

$$= \langle q_0 2^\rho, q_0 x_1 - q_1 x_0, \ldots, q_0 x_t - q_t x_0 \rangle \tag{3.3}$$

Since $x_i = q_i p + r_i$, the terms simplify:

$$q_0 x_i - q_i x_0 = q_0(q_i p + r_i) - q_i(q_0 p + r_0) = q_0 r_i - q_i r_0 \tag{3.4}$$

This cancellation of p means $\vec{v}$ is a relatively short vector in the lattice L, with length roughly $2^{\gamma+\rho-\eta}\sqrt{t+1}$. If a lattice reduction algorithm can find this target vector $\vec{v}$, the attacker recovers the quotients, and subsequently recovers $p = \lfloor x_0/q_0 \rceil$.

The Climax of Part 3: Defeating LLL "So, we have a target vector $\vec{v}$ in a lattice. Can LLL or BKZ find it? This is where the paper's parameter selection shines.
By Minkowski's theorem, we know there are vectors in this lattice bounded by the determinant. If we set γ (the bit-length of the public key elements) to be massively large—specifically $\gamma = \omega(\eta^2 \log \lambda)$—two incredible things happen to protect the scheme.
First, if the attacker uses a small number of samples t, the lattice dimension is small, but Minkowski's bound tells us there are exponentially many spurious vectors in the lattice that are much shorter than our target $\vec{v}$. The target is buried in geometric noise.
Second, if the attacker uses a huge number of samples t to make $\vec{v}$ the absolute shortest vector, the dimension of the lattice becomes massive. Known lattice reduction algorithms take time roughly $2^{t/k}$ to find a 2^k approximation of the shortest vector. To get an approximation tight enough to isolate $\vec{v}$, the algorithm would require time roughly $2^{\gamma/\eta^2}$. Because we set $\gamma = \omega(\eta^2 \log \lambda)$, this time is super-polynomial. The attack is thwarted."

Summary of Secure Parameters: To ensure security against brute force, factoring, and SDA lattice attacks, the authors establish:

- $\rho = \mathcal{O}(\lambda)$ (prevents brute force on the noise)
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$ (supports homomorphic depth, to be explored next)
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$ (thwarts lattice reduction)

While this results in large public keys (elements are $\tilde{\mathcal{O}}(\lambda^5)$ bits long), it guarantees rigorous security based solely on the hardness of the Approximate-GCD over the integers.